

Execution: Data, Modeling, Deployment & Documentation

Overview

Capstone execution should mirror real AI engineering workflows: data pipeline, modeling loop, deployment path, monitoring plan, and professional documentation. The objective is not just a model score; it is a defensible system narrative.

Execution sequence:

1. clarify requirements and constraints,
2. establish data and baseline pipelines,
3. iterate models and evaluations,
4. package and deploy (or realistically simulate deployment),
5. document decisions and outcomes.

Data & Modeling

Data Workflow Expectations

A capstone-quality data workflow includes:

- source documentation and provenance,
- cleaning and schema checks,
- train/validation/test split discipline,
- feature/embedding pipeline reproducibility,
- assumptions and limitations.

Modeling Workflow Expectations

- baseline first, then incremental improvements,
- metric choice tied to business/research objective,
- error analysis across segments,
- calibration/threshold decisions where relevant,
- robustness checks (drift, sensitivity, stress).

Applying Earlier Lessons

- ML basics for controlled experimentation.
- DL/NLP/CV modules for domain-specific tasks.
- GenAI/RAG modules for retrieval and LLM-based workflows.
- Safety modules for guardrails and risk checks.

Deployment & MLOps/LLMOps Integration

Deployment Strategy Choices

Choose one path appropriate to scope:

- **Batch:** scheduled inference and report generation.
- **Online API:** request/response serving with simple auth and logging.
- **Streaming/Event-based:** when continuous ingestion is essential.

Capstone-Scale MLOps/LLMOps Minimum

Even in lightweight projects, include:

- versioning of model/code/config,
- reproducible environment notes,

- minimal telemetry (latency, errors, key metrics),
- rollback or fallback behavior,
- retraining/update strategy statement.

LLM-Centric Projects

For RAG/agent capstones, add:

- prompt and retrieval versioning,
- eval sets for hallucination/groundedness,
- safety filters and refusal behavior,
- token/cost tracking.

Documentation & Artefacts

Core Deliverables

A strong capstone package typically contains:

- repository with runnable code,
- architecture diagram and flow narrative,
- model/evaluation report,
- deployment guide and operations notes,
- demo script or short walkthrough,
- reflection on trade-offs and risks.

Reproducibility Checklist

- pinned dependencies,
- seed setting and random-state notes,
- data access assumptions,
- command sequence for training and serving,
- expected outputs.

Technical Storytelling

Documentation should answer:

- what problem was solved,
- why this architecture was chosen,
- what evidence supports quality,
- what remains risky or future work.

Capstone Case Studies & Exceptions

Case 1: Deep Learning Capstone to Product Prototype

A vision model project started as experimentation but shipped a minimal API and dashboard. Strong documentation made it portfolio-ready despite modest model novelty.

Case 2: Research-Heavy Capstone

A causal ML capstone focused on method comparison and ablations, with limited deployment. The team produced high-quality experiment records and clear reproducibility scripts, which still signaled strong engineering maturity.

Case 3: LLM Workflow Capstone

A RAG assistant project achieved good demo quality but initially lacked observability. Adding structured logs, evaluation sets, and safety checks significantly improved professional credibility.

Exceptions

- Full cloud infra is optional if local reproducible simulation is strong.
- If deployment is infeasible, compensate with rigorous architecture and runbook documentation.

Interview Questions & Answers

1. **Q:** Describe your capstone workflow end-to-end. **A:** Problem definition, data pipeline, baseline modeling, iterative evaluation, deployment packaging, monitoring design, and documentation.
2. **Q:** How did you validate model quality? **A:** Baseline comparison, holdout metrics, segment-level error analysis, and sensitivity checks.
3. **Q:** Why include a baseline model? **A:** It establishes reference value and prevents over-claiming improvements.
4. **Q:** How did you choose metrics? **A:** Metrics were mapped to project risk and stakeholder goals.
5. **Q:** What deployment pattern did you choose and why? **A:** Chosen based on latency, complexity, and maintenance constraints.
6. **Q:** How did you handle reproducibility? **A:** Fixed seeds, pinned deps, explicit scripts, and config tracking.
7. **Q:** What MLOps elements did you include? **A:** Versioning, artifact tracking, basic monitoring, and update plan.
8. **Q:** How did you monitor performance post-deployment? **A:** Logged predictions/latency/errors and defined threshold alerts.
9. **Q:** What if your best model was too heavy for serving? **A:** Served a lighter model and documented performance-latency trade-offs.
10. **Q:** What was your biggest engineering trade-off? **A:** Balancing model complexity with delivery reliability and timeline.
11. **Q:** How did you test for data issues? **A:** Schema checks, missingness analysis, leakage checks, and drift proxies.
12. **Q:** What documentation was most valuable? **A:** Architecture and decision logs that explain why choices were made.
13. **Q:** Did you include security/safety controls? **A:** Yes, through scoped access, input validation, and risk guardrails.
14. **Q:** How did you structure your repo? **A:** Separate modules for data, training, serving, and docs with clear entrypoints.
15. **Q:** What would you improve in a second version? **A:** Stronger automated tests, richer monitoring, and CI-driven deployment.
16. **Q:** How do you justify deployment in a student project? **A:** A minimal serving path demonstrates real operational readiness.
17. **Q:** How did you communicate limitations? **A:** Explicit limitations section with mitigation and next-step plan.
18. **Q:** What if deployment was out of scope? **A:** Provide API stub plus runbook and architecture proof-of-feasibility.
19. **Q:** How did you connect the project to business value? **A:** Converted technical outputs into KPI/impact estimates and risk narrative.
20. **Q:** What artefacts would you show in an interview? **A:** Repo, architecture diagram, evaluation report, demo, and postmortem notes.

Source-Informed References

- IBM AI Engineering Professional Certificate: <https://www.coursera.org/professional-certificates/ai-engineer>
- IBM AI Capstone Project with Deep Learning: <https://www.coursera.org/learn/ai-deep-learning-capstone>
- South Dakota AIE 465 Capstone II (implement, test, validate, deploy): <https://www.sdstate.edu/sites/default/files/file-archive/2026-04/AIE%20465%20Artificial%20Intelligence%20Engineering%20Capstone%20II%20%28NCR%29.pdf>
- Monash FIT3193 AI project lifecycle and documentation: <https://handbook.monash.edu/2026/units/fit3193>
- Penn State AI program culminates in A-I 894 capstone: <https://psu-public.courseleaf.com/graduate/programs/majors/ai-intelligence/>
- MLOps & LLMOps production workflow course context: <https://www.coursera.org/learn/mlops-and-llmops-deploying-and-scaling-ai-in-production>